Multimedia Tools and Applications (2024) 83:62209–62235
https://doi.org/10.1007/s11042-023-16441-3

1229: MULTIMEDIA DATA ANALYSIS FOR SMART CITY ENVIRONMENT SAFETY

Pose4Gun: A pose-based machine learning approach to detect small firearms from visual media

Rajdeep Chatterjee[1,2] · **Ankita Chatterjee**[2]

Received: 3 September 2022 / Revised: 20 June 2023 / Accepted: 28 July 2023 /
Published online: 5 September 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
Violence due to firearms is a menace and is growing across the globe. Mostly small firearms such as pistols and revolvers are used in close proximity for violence in public places. Our moral duty is to contribute to solving this problem with our expertise as it is impossible to keep 24 × 7 eyes on everyone in the street and corners. An automatic lightweight artificial intelligence solution to detect such small firearms is a welcoming step towards a more secure society. Here, we have proposed a hand pose pattern analysis to identify whether a person is holding a pistol or revolver. Furthermore, different data prepossessing, machine learning and deep learning techniques have been implemented to classify guns and no-gun from visual media. Finally, we have introduced a metric learning approach instead of classification along with our novel Fuzzy Discernible Feature Selection (FDFS) technique to provide faster and more accurate discrimination between gun and non-gun instances from images and videos. The result from Deep Neural Network + FDFS has achieved 93% test accuracy and outperforms the other used machine learning and deep learning alternatives.

Keywords Metric learning · Image classification · Firearms detection · Pose estimation

1 Introduction

The small firearms indicate either "a small gun held in and fired from one hand. It can be a loaded one or an automatic pistol"[1] or "a type of small gun held in one hand that can be

[1] Pistol: https://dictionary.cambridge.org/dictionary/english/pistol

Rajdeep Chatterjee and Ankita Chatterjee are contributed equally to this work.

✉ Rajdeep Chatterjee
cse.rajdeep@gmail.com

Extended author information available on the last page of the article

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

fired several times without putting more bullets in it (mostly six bullets)"[2]. Gun violence is rampant across the globe and a significant threat to the civil and liberal world. Every day, approximately 2000 people are injured by gunshots. People living with firearm injuries are estimated at least two million [1]. More than 250 mass shootings have happened till June 2022 in the United States [2]. As per US Public Law 112-265 112th Congress (2013), *mass killings*[3] due to gun violence is defined as a single shoot-out in a public place where the victims are more than three or four excluding the shooters. Besides mass killings, suicide using guns is also a significant social issue according to Pew Research [3]. Suicides alone contributed more than 50% of the total number of United States gun-related deaths in 2020.

However, the US Congress has passed legislation introducing stricter gun ownership rules in America. This firearms legislation is a welcome step to impose restrictions on buyers below the age of 21, people having some mental illness, cancel firearms of convicted abusers, and people termed as a threat by law enforcement agencies [4, 5]. Gun violence is also on the rise in India. The murder of Indian Singer Sidhu Moose Wala on 29 May 2022 in his car by unidentified shooters raised the issue of gun culture in India [6]. We hope that the Government of the day imposes more restrictions on gun licenses and usage. Artificial Intelligence, specifically the deep-learning-based automatic detection of guns from CCTV, is considered a solution for preventing unfortunate gun violence [7, 8]. Different state-of-the-art (SOTA) Convolutional Neural Network (CNN) models [9–12] are commonly used for classification and Faster RCNN, YOLO variants have been employed for object detection problems [13, 14]. However, deep learning carries a caveat: it requires a huge amount of input data and computation power to build a decent classification or detection model. The contributions of this paper can be summarized bellow:

- Explore the recent advancements in CPU-driven pose estimation of different body parts to provide an opportunity to implement simple pose-based gun detection from an image or video.
- The authors have proposed pose estimation data of human hands (hand landmarks) to generate the input data
- Furthermore, a suitable feature selection technique and classifier model for an efficient classification between gun and non-gun images.

The paper is divided into six sections. The related work has been discussed in Section 2. In Section 3, the theoretical concepts have been described. The following Section 4 explains the dataset and the experimental setup. The results are explained in Section 5. Finally, we conclude the paper in Section 6.

2 Related work

Gun detection using computer vision technology in public places is crucial for ensuring the safety and security of individuals. The use of firearms for criminal activities such as shootings has become a major concern in many countries, and it's crucial to identify and stop such incidents as quickly as possible. Computer vision-based gun detection technology enables public places such as schools, airports, and malls to quickly identify firearms and alert authorities in real time, allowing for swift action to be taken to prevent potential harm. Moreover, computer vision technology can be used to identify and track potential suspects

[2] Revolver:https://dictionary.cambridge.org/dictionary/english/revolver

[3] Mass Killings: https://www.congress.gov/112/plaws/publ265/PLAW-112publ265.pdf

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

or perpetrators by analyzing their behavior, facial recognition, or license plates. This can aid in the investigation and prosecution of criminal activities involving firearms.

In literature, there are different deep learning-based approaches for weapon detection techniques [15]. The authors propose a deep ensemble model. It splits the captured gun image parts. Then, the proposed deep neural network is used to detect and locate the gun. Eventually, the outcome has been obtained through the aggregation of its different outcomes [16]. Here the issue is that the authors use only one type of rifle, i.e., the AR-15. A comparative analysis of their model is missing. In [17], authors have proposed a YOLOV5-based gun detection technique and empirically proved that it outperforms traditional Faster RCNN with VGG16 and ResNet50 backbone architectures. The paper also highlights the different deep-learning approaches in its related work section. Nowadays, researchers have explored body pose estimation for detecting whether a person is posing with a gun or not. One such paper is [18]. Authors have proposed an improvement of an average precision from 4.23 to 18.9 over previous models on the same dataset. Full-body pose estimation and a CNN model are used to detect guns. However, it takes a longer training time. In [19], authors have proposed a methodology that first determines the hand regions guided by the human pose estimation and then analyzes those regions using a weapon detection model. For optimal localization of each hand region, we defined a new factor, called the Adaptive pose factor, that considers the body's distance from the camera. In [20], authors have introduced an improvement over a handgun detector by leveraging the human pose. The proposed network provides successful results, with a maximum improvement of 17.5% in AP of the proposed combinational model over the baseline handgun detector (YOLOv3). Our work shows that the individual's pose can be effectively used to improve threat detection accuracy. However, there is a threshold value to adapt the overall detection accuracy. The choice of the optimal value for different scenarios is a drawback of the model.

3 Theoretical background

3.1 Pose estimation

MediaPipe [21–23] is a lightweight, efficient multi-modal machine learning open-source application developed by Google, works on different platforms such as Android, iOS, and the web. The Python version of MediaPipe can be downloaded from *PyPI.org* for Linux, Windows, and Mac-OS. MediaPipe gives an option for customizing the prebuilt framework based on one's requirements. MediaPipe Hands is an advanced off-the-shelf hand and finger detection and tracking library[4]. It provides 21 ($\times 3$: x, y, z axes) keypoints of a hand from a single frame[5]. The x and y values are normalized to $0.0-1.0$ based on image width and height. Moreover, the z-axis is the relative depth working on the principle of weak perspective. Weak perspective projection tries to approximate the perceived projection by keeping the wrist as a root coordinate with the assumption that all hand keypoints of a 3D object are at roughly the same distance from[6] the camera (Fig. 1).

[4] MediaPipe: https://google.github.io/mediapipe/

[5] MediaPipe Hands: https://google.github.io/mediapipe/solutions/hands

[6] Hand landmarks: https://google.github.io/mediapipe/solutions/hands.html

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

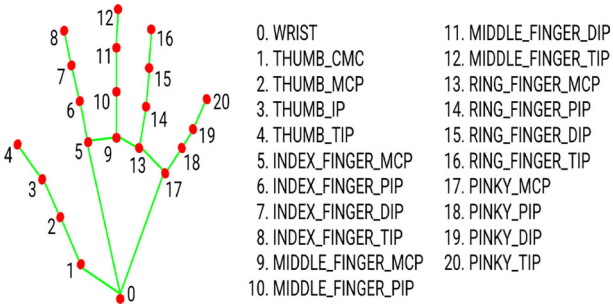


Fig. 1 Mediapipe hand landmarks

3.2 Feature selection

3.2.1 Genetic algorithm feature selection

A genetic algorithm is an evolutionary technique to find optimal solutions in search space. Here, Binary Genetic Algorithm has been used for feature selection. The chromosomes used in the population consist of either 0s or 1s. The length of the chromosomes is the same as the dataset's actual feature-set size. An one in i^{th} gene (one value of chromosomes) indicates the corresponding (i^{th}) feature has been selected, and zero suggests rejection. Now on, the process is termed Genetic Algorithm Feature Selection (GAFS) [24, 25].

N is the number of individuals in the population; Cr is the crossover rate ($0 < Cr < 1$) in each iteration; and μ is the mutation rate [26, 27]. A brief outline of the Genetic Algorithm has been given in Algorithm 1.

Algorithm 1 : GA(n, Cr, μ).

Input: k: 0
P_k: a population of N randomly-generated individuals
Output: the fittest individual from P_k^*
Evaluate P_k
repeat
  /* Create generation k + 1 */
  /* Select parents from population */
  Select $(1 - Cr) \times N$ members of P_k and insert into $P_k + 1$
  /* Crossover and produce new population */
  Select $Cr \times N$ members of P_k
  pair them up
  produce offspring
  insert the offspring into $P_k + 1$
  /* Mutation on new population */
  Select $\mu \times N$ members of $P_k + 1$; invert a randomly-selected bit in each
  /* Evaluate fitness of population $P_k + 1$ */
  Compute fitness(i) for each i $\in P_k$
  /* Update k */
  k ← k + 1
until *Convergence or Maximum iterations (max_iter)*

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3.2.2 Particle swarm feature selection

Particle Swarm Optimization is inspired by the flocking of birds. Both continuous and binary-valued particles have been used. As the feature selection task, the particles use 0s or 1s only. The dimension of a particle equals the original feature-set size of the given dataset. Here on, it is mentioned as Particle Swarm Feature Selection (PSFS) [28].

The Particle Swarm Optimization has two central equations for updating each of the particles in a population [29, 30]. These are given below:

$$V_{k+1}^{ij} = wV_k^{ij} + c_1 r_1 (pbest_k^i - P_k^{ij}) + c_2 r_2 (gbest_k - P_k^{ij}) \tag{1}$$

$$P_{k+1}^{ij} = P_k^{ij} + V_{k+1}^{ij} \tag{2}$$

where, i = 1,2,. . .,P (P, population size) and j = 1,2,. . .,n (n, dimension). The term w is inertia weight; its value is taken from $0 \leq 1$. The parameter c_1 indicates the individual-cognition parameter and suggests the weight of a particle's past experiences. Similarly, c_2 suggests the social learning parameter, which provides the importance of the experience of the swarm through global learning. Both c_1 and c_2 values are usually used as a positive constant ([1, 2]). The other parameters that are r_1, and r_2. These are random values from [0, 1]. It helps in introducing exploration while searching for the optimal solution. A brief overview of the Particle Swarm Optimization has been given in Algorithm 2.

Algorithm 2 : PSO(N, w, c_1, c_2).

Input: k: 0
  P_k: N randomly-generated individuals
  V_k: N randomly-generated velocities for P_K
Output: the fittest individual from P_k^*
/* Initialization of Population, k=0 */
Evaluate P_0
while *each* $i \in P_0$ **do**
  Evaluate the fitness P_0^i
  $pbest_0^i \leftarrow$ fittest value for i^{th} particle (of P_0^i)
end
$gbest_0 \leftarrow$ best fitness value from particles, P_0
repeat
  while *each* $i \in P_k$ **do**
    Update P_k^i and V_k^i using (1) and (2)
    Evaluate the fitness P_k^i
    $pbest_k^i \leftarrow$ fittest value for i^{th} particle (of P_k)
    if $\mathcal{F}(P_k^i) < \mathcal{F}(pbest_k^i)$ **then**
      $pbest_k^i \leftarrow P_k^i$
    end
    if $\mathcal{F}(P_k^i) < \mathcal{F}(gbest_k)$ **then**
      $gbest_k \leftarrow P_k^i$
    end
  end
  k ← k+1
until *Convergence or Maximum iterations (max_iter)*
$P_k^* \leftarrow gbest_k$

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3.2.3 Gravitational search feature selection

Gravitational Search Feature Selection (GSFS) is Gravitational Search Algorithm (GSA) optimization technique-based feature selection method [31, 32]. The GSA can be identified as a system of small objects of masses in an artificial universe obeying the Newtonian laws of gravitation and motion. More precisely, the masses obey the following law: "*Every particle in the universe attracts every other particle with a force that is directly proportional to the product of their masses and inversely proportional to the square of the distance between them*"

$\hat{k}$best is a function of time, with the initial value K_0 at the beginning and decreasing with time. In such a way, in the beginning, all agents apply the force, and as time passes, $\hat{k}$best is decreased linearly, and at the end, there will be just one agent (fittest) applying force to the others. It is an "Elitist Search" that balances between exploration and exploitation in an optimization problem [33]. The overview of the Gravitational Search Algorithm has been given in Algorithm 3.

$$G_k = G_0 e^{-\alpha \frac{t}{T}} \tag{3}$$

$$best_k = min(\sum_{i=1}^{N} \mathcal{F}_k^i \tag{4}$$

$$worst_k = max(\sum_{i=1}^{N} \mathcal{F}_k^i) \tag{5}$$

$$m_k^i = \frac{\mathcal{F}_k^i - worst_k}{best_k - worst_k} \tag{6}$$

$$M_k^i = \frac{m_k^i}{\sum_{y=1}^{N} m_k^y} \tag{7}$$

$$a_k^i = \frac{F_k^{ij}}{M_k^i} \tag{8}$$

$$v_{k+1}^{ij} = r_i v_k^i + a_k^i \tag{9}$$

$$x_{k+1}^{ij} = x_k^{ij} + v_k^{ij} \tag{10}$$

The traditional GSA and PSO work on continuous values. Nevertheless, it has been adapted to a binary GSA, and binary PSO for feature selection technique which consists of either 0s or 1s [34]. Sigmoidal function ($\mathcal{S} = \frac{1}{1+e^{-z}}$) has been used with 0.5 as a threshold value. We pass a real-value (z) that is an element of a particle (solution vector) through $\mathcal{S}$, and if the outcome is greater than the given threshold, it is set to 1 otherwise 0. The dimensions of the GSA agents equal the actual feature-set size of the given dataset (refer to Fig. 2).

3.2.4 Quick-reduct feature selection

Often, available information is imprecise, incomplete, and uncertain. It needs to be processed the uncertainty in the given data. In the 1980's, *Rough Set Theory* was proposed by Zdzislaw Pawlak [35, 36]. Pawlak introduces an equivalence relation based on the attribute-indiscernibility relation. A set is said to be "rough" if its boundary region is non-empty;

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Algorithm 3 : GSA(N, $\hat{K}$best, G_0).

Input: k: 0
P_k: N randomly-generated individuals
v_k: N randomly-generated velocities for P_K
G_0: gravitational constant (changes over iterations)
$\hat{k}$best: first $\hat{k}$ agents with the best fitness value
Output: the fittest individual from P_k^*
/* Initialization of Population, k=0 */
Evaluate P_0
Compute $best$, $worst$, M and a for P_0
repeat
  while *each* $i \in P_k$ **do**
    Evaluate the fitness P_k^i
    Update G, $best$ and $worst$ of the population P_k using (3), (4) and (5)
    Compute M and a using (7) and (8)
    Calculate X_0 (position) and V_0 using (9) and (10)
  end
  k ← k+1
until *Convergence or Maximum iterations (max_iter)*

otherwise, the set is crisp. Lower approximation and Upper approximation have been developed to compute the Positive region. There are usually several subsets of minimal features, and the respective γ value is as close as 1, called reducts. The popular rough-set theory-based reduct identification technique is known as Quick-Reduct. Henceforth, the feature selection technique is termed QRFS.

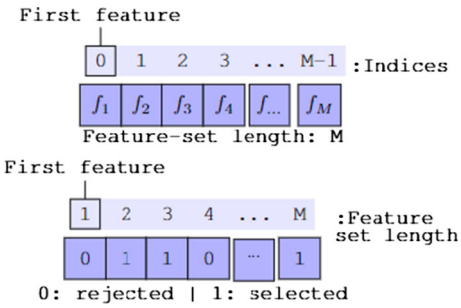
Assume, IS = (U,A) be an information system, then with any $B \subseteq A$ there is an associated equivalence relation [37] in (11):

$$IND_{IS}(B) = \{(x, x') \in \cup | \forall a \in B, a(x) = a(x')\} \tag{11}$$

where $IND_{IS}(B)$ is called the B-indiscernibility relation. (x, x') If then objects x and x' are indiscernible from each other by attributes from B. The equivalence classes of the B-indiscernibility relation are denoted by $[x]_R$.

Let, $B \subseteq A$ and $X \subseteq \cup$. We can approximate X using only the information contained in B by constructing the B-lower (30) and B-upper approximations (31) of X, denoted $\underline{B}X$ and $\overline{B}X$ respectively. B-boundary region of X (32) consists of those objects that we cannot decisively classify into X in B. B-outside region of X (33) consists of those objects that can be with certainty classified as not belonging to X. A set is said to be "rough" if its boundary

Fig. 2 Binaraized PSO and GSA



Content courtesy of Springer Nature, terms of use apply. Rights reserved.

region is non-empty, otherwise the set is crisp.

$$\underline{B}X = \{x|[x]_\mathsf{B} \subseteq X\} \tag{12}$$

$$\overline{B}X = \{x|[x]_\mathsf{B} \cap X = \emptyset\} \tag{13}$$

$$BN_B(X) = \overline{B}X - \underline{B}X \tag{14}$$

$$Out_B(X) = \cup - \overline{B}X \tag{15}$$

Let D and C be subsets of A. We will say that D depends on C in a degree k ($0^{\log}_{\neq} k^{\log}_{\neq} 1$) denoted by $C \Rightarrow_k D$ in (34) if

$$k = \gamma(C, D) = \frac{|\mathsf{POS}_C(D)|}{|\cup|} = \sum_{X \in \cup/D} \frac{|\underline{\mathsf{C}}(\mathsf{X})|}{|\cup|} \tag{16}$$

where $POS_c(D) = \bigcup_{X \in \cup/D} \underline{C}X$ called C-positive region of D.

QuickReduct algorithm [38, 39] is the degree of dependency of that attribute which is not equal to zero. The algorithm tries to find a minimal reduct without generating all possible subsets. Initially, we take an empty set and add in the empty set R those attributes that will result in the most significant increase in dependency value one by one until we get the maximum possible value for the dataset.

Algorithm 4 : QuickReduct($\mathbb{A}$,$\mathbb{C}$).

```
Input: 𝔸: set of all conditional attributes
       ℂ: set of decision attributes
Output: ℝ: set of reduced attributes
R ← ∅ ;                                          // assign an empty set
/* run a loop until condition is failed                                */
repeat
    P ← R ;                                          // assign R to P
    ∀a ∈ (𝔸 − R) ;                                // select a from (A-R)
    if γ_{R⋃x}(ℂ) > γ_P(ℂ) then
        /* assign R and attribute x to P                               */
        P ← R⋃a
    end
    R ← P ;                                          // assign P to R
until γ_R(ℂ) ≤ γ_𝔸(ℂ)
```

$$[x]_A = IND(\{A\}) = \{\{x_1, x_4\}, \{x_2, x_3\}\} \tag{17}$$

$$[x]_B = IND(\{B\}) = \{\{x_1, x_2\}, \{x_3, x_4\}\} \tag{18}$$

$$[x]_C = IND(\{C\}) = \{\{x_1, x_2, x_3, x_4\}\} \tag{19}$$

$$[x]_{AB} = IND(\{AB\}) = \{\{x_1\}, \{x_2\}, \{x_3\}, \{x_4\}\} \tag{20}$$

$$[x]_{AC} = IND(\{AC\}) = \{\{x_1, x_4\}, \{x_2, x_3\}\} \tag{21}$$

$$[x]_{BC} = IND(\{BC\}) = \{\{x_1, x_2\}, \{x_3, x_4\}\} \tag{22}$$

$$[x]_{ABC} = IND(\{ABC\}) = \{\{x_1\}, \{x_2\}, \{x_3\}, \{x_4\}\} \tag{23}$$

Let, $D = \{x \mid Class(x) = \text{yes or no}\}$. The yes and no are denoted as Y and N in Table 1. So, $D = \{\{x_1, x_4\}, \{x_2, x_3\}\}$. Now, $\underline{A}D$ | where D:Y $\Rightarrow \{x_1, x_4\}$ and $\underline{A}D$ | where D:N

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

$\Rightarrow \{x_2, x_3\}$. Therefore, (34) can be rewritten as,

$$k = \gamma(A, D) = \frac{|\bigcup \underline{A}(X)|}{|\cup|} = \frac{|\{x_1, x_4\} \cup \{x_3, x_4\}|}{|\{x_1, x_4, x_3, x_4\}|} = 1 \tag{24}$$

Similarly, $\gamma(B, D) = \gamma(C, D) = 0$ which can also be described as $BN_B(X) = BN_C(X) = \{x_1, x_4, x_3, x_4\}$. If $k = 1$ we say that X depends totally on A. If k < 1 we say that X depends partially (in a degree k) on A. Here, $k = 1$ suggests that the attribute "A" alone is containing good enough discernible information over the dataset given in Table 1. Diagrammatic representation of the Set Approximation for attribute A from Table 1 is shown in Fig. 3.

3.2.5 Fuzzy discernible feature selection

Different types of univariate and multivariate feature selection techniques are available in the literature. The former [40] are typically fast but provide sub-optimal solutions, whereas the latter [41] is too time-consuming. All the features mentioned above in selection techniques are multivariate. We prefer univariate feature selection techniques for their lightweight computational requirements. Classical Discernibility Matrix (DM)-based feature selection is one of the simple and less computation-intensive techniques compared to other rough set-based variants [42]. In Table 1, a toy dataset has been given to explain DM. DM is a symmetric matrix obtained from it (refer Table 2). It uses discrete values, and if the dataset contains continuous values, it requires a suitable binning technique to discretize the dataset. The DM aims to find such features for which there exists discernibility between any two samples. All the single entries are considered as *CORE* and the minimal subset, which includes all the CORE features, is called *REDUCT*. Here, $Core = \{A\}$, $Reduct = \{A, B\}$ (as feature C does not contain any discriminating information between the Y and N classes) [43, 44].

Let, $DT = (X, A, D)$ be a decision dataset with Universe $X = \{x_1, x_2, \ldots, x_m\}$ (where m indicates the total number of instances in the dataset), conditional attributes (features) A and decision class D. Here, $c(x_i)$ and $c(x_j)$ are the discretized data of the feature values x_i and x_j for the ith and j^{th} trials respectively. The c is a feature from the original feature-set A included in the c_{ij} based on the discern-equivalence condition. $d(x_i)$ and $d(x_j)$ indicate the decision class types of the ith and j^{th} trials respectively. Note that $d(x_i), d(x_j) \in D$. The discernibility matrix of DT is denoted by $DM(DT)$, where each cell entry can be defined by discern-equivalence condition as follows in (25):

$$c_{ij} = \begin{cases} \{c \in A : c(x_i) \neq c(x_j)\} & \text{if } \ d(x_i) \neq d(x_j) \\ \emptyset & \text{if } \ d(x_i) = d(x_j) \end{cases} \tag{25}$$

where $i, j = 1, 2, \ldots, m$.

Table 1 Example: Toy Dataset

Samples	A	B	C	Class
x_1	2	6	7	Y
x_2	10	6	7	N
x_3	10	15	7	N
x_4	2	15	7	Y

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

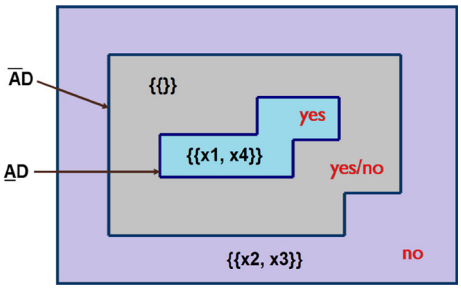


Fig. 3 Pictorial Depiction of Rough Set Approximation for attribute A from Table 1

In (25), c_{ij} is the set of all the discernible features which discriminate instances x_i and x_j into different decision classes. All the single entries are called *Core* (where, $|c_{ij}|$=1) [45, 46].

A new *Fuzzy Discernibility Matrix*-based feature selection algorithm *Fuzzy Discernible Feature Selection* (FDFS) has been used here. FDM works with continuous, that is, a real-valued dataset. The primary motivation behind developing the novel FDM algorithm is to use the real continuous values (without discretization, thereby eliminating information loss) and retain the simplicity of the traditional Discernibility Matrix-based feature selection technique. A detailed discussion on FDFS can be found at [43]. However, a few necessary aspects have been briefly explained in this paper. The 1-Dimension Gaussian Membership (see in (26)) function has been used for the fuzzification of a given dataset.

$$f(\vec{X}; \sigma_p, c_p) = e^{(\frac{-(x_j-\mu_p)^2}{2\sigma_p^2})} \tag{26}$$

Here, $\vec{X} = \{x_j\}$ is the non-fuzzy feature value where $j = \{1, 2, \ldots,$number of instances$\}$. The σ_i and μ_i are the standard deviation and mean of the ith fuzzy set respectively. Assume $D_A = (X, A \cup D)$ is a given dataset with Universe $X = \{x_1, x_2, \ldots, x_m\}$ (where m indicates the total number of samples in the dataset), conditional attributes (crisp features) A and binary decision class $D = \{L, R\}$. After fuzzification, D_A transforms to F_D as explained earlier. In F_D, a crisp feature $c \in A$ decomposed to f_c^L and f_c^R fuzzy values[7] using (26). For a set c, denote the fuzzified set as c^*. Let $d(x_i)$ and $d(x_j)$ indicate the decision class types of the ith and j^{th} trials respectively. The proposed fuzzy discernibility matrix of D_A is denoted by $FD_M(F_D)$, where each cell entry is a vector of size $n = |A|$ defined by fuzzy discern equivalence condition (that is, dissimilarity measure) as follows in (27):

$$c_{ij} = \begin{cases} (\mathcal{D}(c^*(x_i), c^*(x_j)), \ \forall\, c \in A), & \text{if} \quad d(x_i) \neq d(x_j) \\ \emptyset & \text{if} \quad d(x_i) = d(x_j) \end{cases} \tag{27}$$

where, $i, j = 1, 2, \ldots, m$ and $c^* = fuzzifySet(c)$.

In (27), c_{ij} is a vector of feature-wise dissimilarity values. Each component t of the vector c_{ij} is a dissimilarity index value that lies between 0 to 1 and discriminates instances x_i and x_j into two decision classes with different degrees of discernibility [43].

The dissimilarity measure between two trials uses the fuzzy feature sets $\vec{X}$ and $\vec{Y}$ (fuzzy values are derived from a crisp feature, and binary decision classes set the fuzzy sets per feature as two). Different dissimilarity measures (distance metrics) can be used. Here, Cosine

[7] We use the shorter notation to mean the fuzzy set $\{(f_c^L, L), (f_c^R, R)\}$

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Discernibility Matrix of Table 1

Samples	$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{x_3}$	$\mathbf{x_4}$
x_1				
x_2	A			
x_3	A,B	∅		
x_4	∅	A,B	A,B	

distance has been employed where $i = 1,\ldots,N$ in (28):

$$\mathcal{D}(\vec{X}, \vec{Y}) = \left[1 - \frac{\vec{X}.\vec{Y}}{\|\vec{X}\|\|\vec{Y}\|}\right] \tag{28}$$

The $\vec{X}$ and $\vec{Y}$ are vectors indicating the fuzzified values $\mu_X(.)$ and $\mu_Y(.)$ for x^{th} and y^{th} samples ($x \neq y$). The cosine distance (dissimilarity function) (28) is computed from established *Cosine similarity* by subtracting it from 1 [47].

The $Reduct$ is the resultant subset of selected features from the original set of features. Finally, the user input $k^{'}$ is used to select the first $k^{'}$ best discriminating features as $Reduct$.

The procedure can be better understood with an example. Here, we consider the same toy dataset mentioned in Table 2. The class-wise means and standard deviations for each crisp feature A, B, and C are calculated and given in Table 3. Now, using (26), Table 2 is transformed into Table 4. Once we obtain the fuzzified table either from a discrete or real-valued dataset. The fuzzy discernibility matrix can be generated a Table 5. There are three features. So, a possible user input $k^{'}$=2 or 1 (< 3).

The mean discernibility vector is $\frac{(x_2,x_1)+(x_3,x_1)+(x_4,x_2)+(x_4,x_2)}{V=4}$, where V= number of valid cell entries in FDM. The purpose of this permutation in-variance step is to keep the fuzzy discernible values [0, 1]. A zero (0) fuzzy discernible value indicates no discriminating information. Except for zeros, the mean discernibility vector is sorted in descending order and selected first $k^{'}$. Here, $Reduct = \{A\}$ and the reduced table is given below as Table 6.

3.3 Learning algorithms

3.3.1 Classifiers

Logistic Regression (LOGRIG) is a popular binary classification algorithm that uses a logistic function to model the probability of a binary response variable based on one or more predictor variables. It is often used when the response variable is categorical in nature and can take only two possible values.

K Nearest Neighbor (KNN) is a non-parametric algorithm used for both classification and regression tasks. It classifies an observation based on the majority class of its k nearest

Table 3 Computed class-wise means and standard deviations for Table 1

Feature	A_N	A_Y	B_N	B_Y	C_N	C_Y
Mean	10.00	2.00	10.50	10.50	7.00	7.00
Std.	0.00	0.00	6.36	6.36	0.00	0.00

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 4 Fuzzified Table 1 using (27) and (26)

Samples	A_N	A_Y	B_N	B_Y	C_N	C_Y	D
x_1	0.00	1.00	0.61	0.61	1.00	1.00	Y
x_2	1.00	0.00	0.61	0.61	1.00	1.00	N
x_3	1.00	0.00	0.61	0.61	1.00	1.00	N
x_4	0.00	1.00	0.61	0.61	1.00	1.00	Y

neighbors in a given feature space. The choice of k is important, as a small k may lead to overfitting, while a large k may lead to underfitting.

Decision Tree (DTREE) is a popular classification algorithm that involves recursively partitioning the feature space based on a set of rules or decisions. It involves selecting the best feature to split the data at each node, to maximize the information gain or minimize the impurity of the resulting subsets. It is known for its interpretability and ability to handle non-linear relationships.

Extreme Gradient Boosting (XGB) is an ensemble learning algorithm that combines multiple weak models (decision trees) to form a strong predictive model. It optimizes the objective function using a gradient boosting approach, where each subsequent model is trained on the residuals of the previous model, intending to minimize the overall loss function.

Metric learning, specifically Neighborhood Component Analysis (NCA), is a technique used for supervised dimensionality reduction and classification. It learns a distance metric that maximizes the classification accuracy of the training set by reducing the distance between examples of the same class and increasing the distance between examples of different classes. It is often used when the feature space is high-dimensional and the data is sparse.

3.3.2 Pose4GunNet: A DNN model

A Deep Neural Network has been developed for classifying the Pose4Gun dataset into its early and mature types. We call it "Pose4GunNet" which has a total of five layers, including one input and one output layer. The neurons' distribution of the Pose4GunNet is 42 (input), 256, 1024, 512, 64, and 2 (output). The input can vary based on different feature selection techniques. However, the original feature-set length is 42. The decision class labels are 0 and 1. Zero indicates an image without a gun (non-gun), and One suggests an image with a gun. We have used the multi-class cross-entropy loss function and SOFTMAX(.) of two classes at the end. The motivation behind developing the Pose4GunNet is to make a lightweight model with a balanced non-linear mapping capacity. The model can be visualized in Fig. 4. One-

Table 5 Fuzzy Discernibility Matrix obtained from Table 1

Samples	x_1	x_2	x_3	x_4
x_1				
x_2	[1.0,0,0]			
x_3	[1.0,0,0]	∅		
x_4	∅	[1.0,0,0]	[1.0,0,0]	

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 6 Reduced Toy Dataset Table 1 using FDFS

Samples	A	Class
x_1	2	Y
x_2	10	N
x_3	10	N
x_4	2	Y

dimensional batch normalization and ReLU have been employed at every layer to minimize[8] the covariate shift problem. The batch-normalization transforms the intermediate inputs into normal stand distribution [48–50].

For Pose4GunNet, We set the learning rate to 0.001, epochs to 50, and optimizer to the Adam algorithm. In the third and fourth layers, dropout with 0.2 probability has been implemented to invoke model regularization. A bird's eye view of the proposed DNN model is given in Fig. 4 (where # indicates the number of input features).

3.3.3 Pre-trained transfer learning

The SOTA CNN model can be used to train an image classifier. There are two common approaches. Firstly, we can use a pre-trained model and then fine-tune them on the customized dataset. Secondly, we can develop a scratch CNN model and train on the customized dataset from the beginning. The former is also known as Transfer Learning as it has prerequisite knowledge due to its past training on a relatively huge dataset, then uses the knowledge to kick-start the training process. Transfer learning is preferred over scratch models due to its fast convergence. Besides the proposed pose-estimated + machine learning gun detection, ResNet50, MobileNetV2, and EfficientNet-B0 with transfer learning are directly used on the gun and non-gun images to build a classifier [51–53]. The results are obtained using PyTorch GPU framework using cross-entropy loss, Adam optimizer, 0.001 learning rate, and 10 epochs.

3.4 Evaluation metrics

In classification, relying only on accuracy may not give you a fair reflection of model robustness. We have used accuracy, F1-score, and AUC (Area Under the Curve) classification evaluation metrics to assess performance.

The used evaluation metrics have been explained through an example "confusion matrix" in Table 7. In Table 7, TP, TN, FP, and FN suggest True Positive, True Negative, False Positive, and False Negative, respectively. The total number of samples is indicated by "P"; whereas "a+c" and "b+d" denote the actual Positive and Negative samples in the population "P". The classification accuracy and F1-Score are calculated using (29) and (32) ((32) is derived from (30) and (31)). On the other hand, the AUC is computed as the Area Under the $Sensitivity$ and $1 - Specificity$ (TPR-FPR) Curve (see in (33) to (34)) [54].

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{29}$$

$$Precision = \frac{TP}{TP + FP} \tag{30}$$

[8] Visualkeras is a viewer for neural network models: https://github.com/paulgavrikov/visualkeras

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 7 Confusion Matrix

		Actual		
		Positive	Negative	Total
Predicted	Positive	TP (a)	FP (b)	$a+b$
	Negative	FN (c)	TN (d)	$c+d$
	Total	$a+c$	$b+d$	P

$$Recall = \frac{TP}{TP + FN} \tag{31}$$

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN} \tag{32}$$

$$Sensitivity = Recall = \frac{TP}{TP + FN} \tag{33}$$

$$Specificity = \frac{TN}{FP + TN} \tag{34}$$

4 Dataset preparation and experimental setup

4.1 Dataset

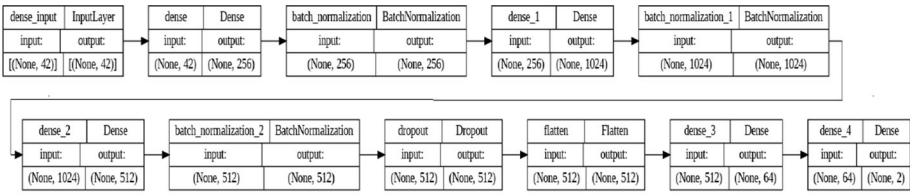
A dataset containing 350 small firearms (pistols or revolvers) has been prepared from a prototype Face and Gun dataset[9] where most of the images are borrowed from the Internet Movie Firearms Database [55]. The dataset has 1851 × 2190 high resolution to 259 × 194 low-resolution images. The raw input images are reshaped to 224 × 224 before using it in the gun detection pipeline. The dataset contains diverse images of small firearms: some have guns in one hand, some have two guns in two hands, and some non-gun images. Some Gun and Non-Gun hand poses samples are shown in Fig. 5.

4.2 Experimental setup

MediaPipe estimated 21 keypoints from each hand. We record the X and Y axes normalized coordinate values (to maintain the scale invariance of an image) in a data frame. The features are stored as follows: Feature-1 and Feature-2 indicate the X and Y axes of hand keypoint 1; similarly, Feature-3 and Feature-4 indicate the X and Y axes of hand keypoint 2, and so forth. There are 42 total features obtained from a single hand (that is, right or left). The dataset provides 227 gun and 196 non-gun samples with 42 features (see in Table 8). We have employed a SMOTE sampler to balance the dataset to 454 samples where each class has 227. The dataset is divided into 80% and 20% for training and testing, respectively. In simple words, we have 91 samples to infer into the gun (1) and non-gun (0) classes. Furthermore, one can observe that the test set has 47 gun and 44 non-gun samples.

[9] Face and Gun: https://github.com/cserajdeep/Face-and-Gun-Dataset

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



(a) Pose4Gun Model Plot

(b) Pose4Gun Model Visualization

Fig. 4 Visualization of the proposed Deep Neural Network (DNN) model (Pose4GunNet): Visualkeras view of the # → 256 → 1024 → 512 → 64 → 2 with 1-Dimensional Batch Normalization (BN) and Dropout at 0.2

Table 8 Results obtained from the different classifiers on the actual dataset without feature selection techniques (Total features: 42)

Model	Acronym	Accuracy	AUC	F1-Score
Logistic Regression	LOGRIG	0.82	0.88	0.83
K-Nearest Neighbor (K=5)	KNN	0.78	0.82	0.82
Decision Tree	DTREE	0.80	0.80	0.82
Extreme Gradient Boosting	XGB	0.88	**0.95**	**0.89**
Metric Learning (NCA)	METRIC	0.81	0.92	0.84
Deep Neural Network	DNN	**0.89**	0.92	**0.89**

(a) Non-Gun Pose-1 (b) Gun Pose-1

(c) Non-Gun Pose-2 (d) Gun Pose-2

Fig. 5 Samples of Gun and Non-Gun hand poses

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

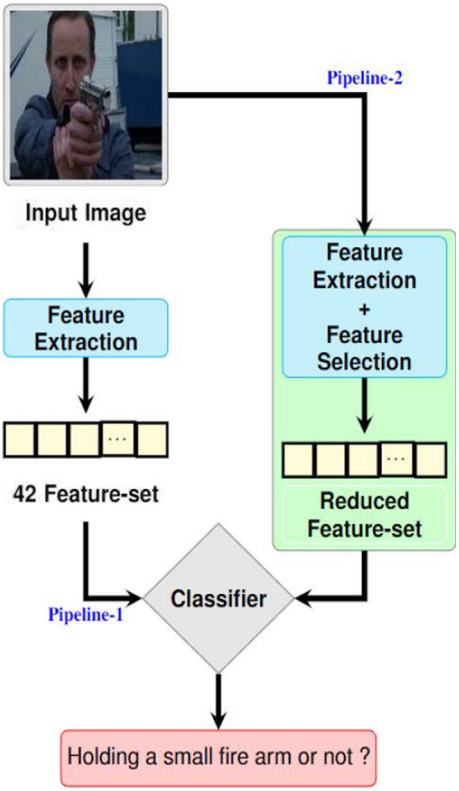


Fig. 6 Visual representation of the proposed frameworks

4.3 Proposed pipeline

It is observed that the fast and lightweight models are well appreciated even in the deep learning-based approaches. Frames per second (FPS) is an important evaluation metric to determine an object detector's correctness. FPS indicates how many frames (images) of video input can be used in any object detection technique. Besides accuracy, we have also focused on inference time (time taken to predict the test set). In this paper, we have introduced a pipeline that combines hand pose estimation, feature selection, and machine learning techniques to efficiently classify images containing persons posed with guns and non-gun objects (e.g., cup, football, hand-sign, etc.).

Hand pose-estimated landmarks are extracted as features through CPU-driven Mediapipe (Mask RCNN), and a derived numerical dataset ($\mathcal{D}'$) has been prepared from the actual image gun/non-gun dataset ($\mathcal{D}$). Once we have the $\mathcal{D}'$, two pipelines are considered for classification. In Pipeline-1, different machine learning models, including a scratch DNN model, have been implemented for classification. On the other hand, in Pipeline-2, various feature selection techniques (GSFS, PSFS, GSFS, QRFS, and FDFS) have been employed to select the best-discriminating features from the original 42 numerical features of $\mathcal{D}'$ (where 42 features obtained from 21 land-marks $\times$ x,y coordinates).

A DNN architecture has been introduced in Section 3.3.2. It is developed from scratch and compared with traditional machine learning and deep learning classifiers based on accuracy and inference time. The proposed pipelines are examined through rigorous empirical means,

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 9 Parameters configuration of different population-based feature selection techniques

Technique	Parameters	Population	Epoch
GAFS	CR=0.80, μ=0.01	10	100
PSFS	w=0.90, c_1=c_2=2.00	10	100
GSFS	G=0.90, α=0.10	10	100

and results have been discussed in the next Section 5. A visual representation of the proposed pipelines is shown in Fig. 6.

4.4 System configuration

The work has been implemented using Python 3.8 on an Intel(R) Core(TM) $i7 - 9750H$ CPU (9th Gen.) 2.60GHz, 16GB RAM and 6 GB NVIDIA GeForce RTX 2060 with 64 bits Windows 10 Home operating system. We have written scratch codes in PyCharm Community IDE.

5 Results analysis and outcomes

The paper only aims to detect small firearms such as pistols or revolvers based on human hand gestures. We have used a lightweight Mediapipe framework and adapted it based on our requirements. It detects hands and also distinguishes between the left and right hands. These X-Y coordinate values are used as features to identify whether the person is holding a gun or non-gun items. We have implemented different object detection techniques to detect firearms in the past, but it takes enormous resources, e.g., memory, GPU, and computation time to train a model. This study bypasses deep learning-based approaches like image classification, object detection, and segmentation. We introduce a framework that trains and infer as fast as possible using the traditional machine learning approaches. Furthermore, different feature selection techniques have been used to identify the best discriminating features (in our case, it is either X or Y coordinates of 21 hand keypoints).

Table 10 Obtained feature-subsets from different feature selection techniques

Feature Selection	Selected Features	Reduct Size	Execution Time (s.)
QR	0,1,2,3,4,5,6,9,10,12,16,17,19, 22,23,24,25,33,34,35,37,41	22	142.82
GA	1,3,9,21,22,23,24,27,32,33,34, 38,39,41	14	14.56
PS	0,1,2,4,5,9,13,16,17,19,21,25 27,28,30,35,36,38	18,	08.90
GS	0,2,4,5,7,8,9,10,11,13,17,19,21, 22,24,25, 27,29,35,36,39	21	25.53
FD	15,13,7,17,9,35,1,11,27,21,23, 5,19,37,3,25,29,31,6,39,33	21	08.29

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

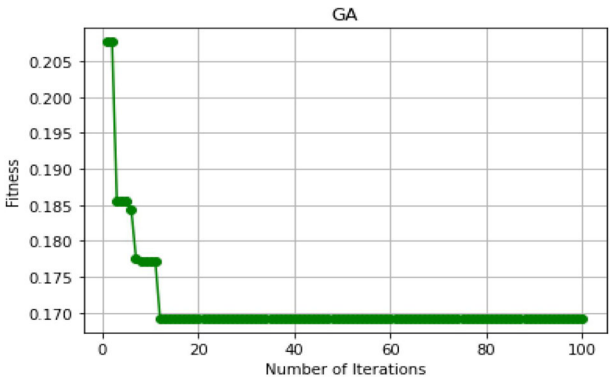


Fig. 7 Performance plot of Genetic Algorithm-based Feature Selection using Metric learning as a evaluating model

Six learners have been implemented to classify the given X-Y coordinate data into either gun (1) or non-gun (0) categories. The performance of these classifiers is shown in Table 8. It is noticed that the DNN outperforms the other alternatives on the original feature-set while we are not employing any feature selection techniques. However, the results are very close to the XGB. The bold-faced entries in the table indicate the best performance.

Rough Set Theory and Equivalence relation-based Quick-Reduct (QRFS) have been used to find important feature subsets. The convergence is as close as 1. In this study, it reaches at most 0.95. Genetic Algorithms with binary encoding chromosomes (GAFS) have been used to select feature subsets with crossover rates and mutation rates of 0.8 and 0.01, respectively. Another two population-based techniques have been implemented: Binary Particle Swarm Optimization (PSFS) and Binary Encoded Gravitational Search Algorithm (GSFS). The PSFS

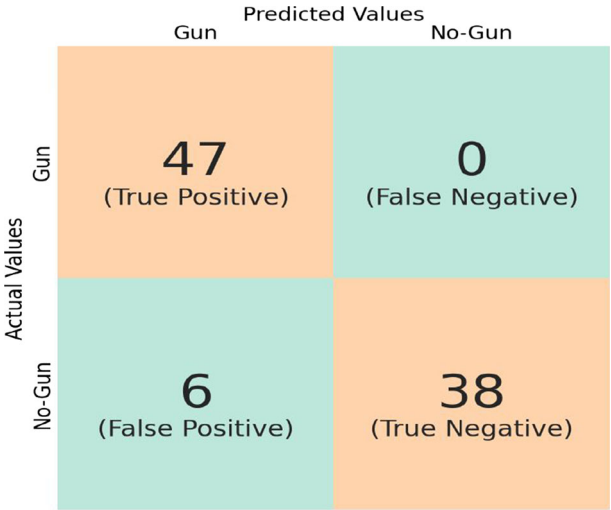


Fig. 8 Confusion matrix obtained from the DNN-based inference on the 91 test-set

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

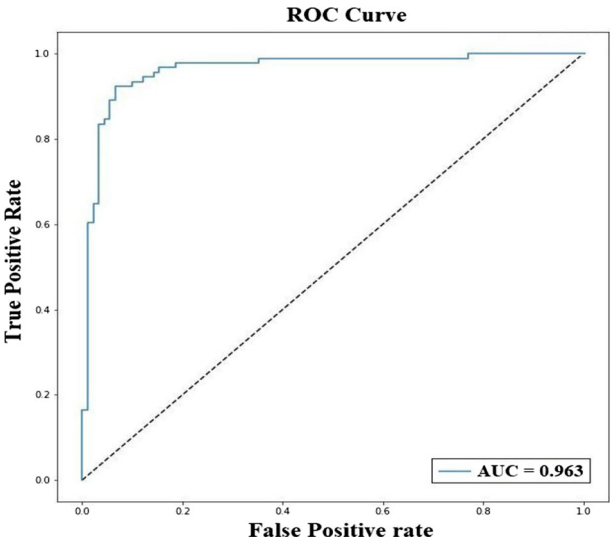


Fig. 9 Area under curve obtained from the DNN-based inference on the 91 test-set

Table 11 Comparative analysis of training-time and inference (test) time in seconds for different models

Technique	Model	Training Time (s.)	Inference Time (s.)
Pose4Gun (Ours)	**LOGRIG**	0.037	0.0004
	KNN	0.010	0.0131
	DTREE	0.021	0.0010
	XGB	0.170	0.0010
	METRIC	0.004	0.0001
	DNN	7.690	0.0134
Image Classification	**ResNet50**	34.27	0.0160
	MobileNetV2	23.51	0.0154
	Efficient-B0	25.78	0.0312
Object Detection	**Faster RCNN VGG16**	3120	0.8100
	Faster RCNN ResNet50	3480	9.3000
	EfficientDet-B0	2400	0.4000

Table 12 Performance obtained from the SOTA CNN Transfer Learning Models

CNN Model	ACC	AUC	F1-SCORE
ResNet50	0.93	0.95	0.93
MobileNetV2	0.91	0.90	0.89
EfficientNet-B0	0.89	0.88	0.84

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 13 Performance obtained from the Logistic Regression classifier on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.80	0.87	0.81
GA	0.79	0.86	0.79
PS	**0.84**	0.88	0.81
GS	0.81	0.88	**0.82**
FD	0.81	**0.89**	0.79

Table 14 Performance obtained from the KNN (K=3) classifier on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.81	0.85	0.84
GA	0.79	0.87	0.82
PS	0.78	0.86	0.81
GS	0.81	0.85	0.84
FD	**0.86**	**0.88**	**0.87**

Table 15 Performance obtained from the Decision Tree classifier on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.84	0.84	0.85
GA	0.84	0.84	0.84
PS	0.85	0.83	0.85
GS	0.77	0.76	0.79
FD	**0.87**	**0.86**	**0.88**

Table 16 Performance obtained from the Metric Learner on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.84	0.94	0.86
GA	0.84	0.93	0.86
PS	0.80	0.93	0.84
GS	0.80	0.91	0.82
FD	**0.89**	**0.93**	**0.90**

Table 17 Performance obtained from the Extreme Gradient Boosting classifier on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.89	0.93	0.90
GA	0.89	0.96	0.91
PS	0.88	0.93	0.88
GS	0.87	0.94	0.88
FD	**0.89**	**0.93**	**0.90**

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 18 Performance obtained from the DNN (Pose4GunNet) on different feature-subsets generated from multiple feature selection techniques

Feature Selection	ACC	AUC	F1-SCORE
QR	0.92	0.95	0.92
GA	0.92	0.96	0.92
PS	0.91	0.95	0.92
GS	0.90	0.94	0.89
FD	**0.93**	**0.96**	**0.93**

has used 0.9, 2, 2 as the inertia w, the constants C_1 and C_2. Again, GSFS uses the constant G as 0.9 and α as 0.1. All three GAFS, PSFS, and GSFS, employ ten population sizes and have been executed for 100 epochs. The summary of the parameter sets can be found in Table 9.

A new univariate feature selection scheme has been used to find out the most discriminating features; it is based on a Fuzzified adaptation of the classical discernibility matrix and reduct computation. Here, we refer to it as FDFS. The $k^{'}$ value is 50% of the actual feature-length, that is 21. The performance of each feature selection schemes on our original 42 features *Gun4Pose* dataset is given in Table 10. It is observed that the QRFS, GAFS, PSFS, GSFS, and FDFS feature subset lengths are 22, 14, 18, 21, and 21, respectively. The noticeable point is that the FDFS has the lowest execution time (8.29 Sec.), and QRFS takes the highest computation time (142.82 Sec.) to generate a reduct. One example plot of GAFS has been displayed in Fig. 7. The fitness function is the equivalent of the error that occurred for each iteration.

The obtained confusion matrix and AUC from the DNN (Pose4GunNet) on the FDFS-based selected features is shown in Figs. 8 and 9.

An interesting empirical observation has been made in Table 11. In Table 11, the machine learning approaches are compared with deep learning-based image classification and object detection models in terms of training and inference time in seconds. However, ResNet50, MobileNetV2 and Efficientnet-B0 have taken 34.27, 23.51, and 25.78 training time (using GPU) respectively. Similarly, the Faster RCNN with VGG16 and ResNet50 backbone is taken 52 Hrs. and 58 Hrs. (using GPU) for training, respectively. EfficientDet-B0 takes 40 Hrs. for training time (using GPU). On the other hand, the machine learning-based models are faster while training and testing (inference). The metric learner approach, which uses a similarity score in the embedding space, performs the fastest training (0.004 Sec.) and inference (0.0001 Sec.) time in our study. The classification results of SOTA CNN models are given in Table 12.

The classification results are evaluated based on accuracy, area under the curve and F1-Score metrics. The performances of all the feature selection techniques for classifiers have been given in Tables 13, 14, 15, 16, 17 and 18.

The visual representations of the proposed outputs are shown in Fig. 10. Figure 10(a)–(c) indicate hand poses. In Fig. 10, (b) and (d) suggest the detected hand poses with (holding a) *gun*. Again, the model has been tested on an unknown to the model video scenes in Fig. 11. it is observed that the proposed model not only detects hand poses while holding a small firearm but also detects the hand.

Finally, Table 19 gives a rigorous summary of the results. FDFS feature selection technique provides 0.875 ± 0.039, 0.908 ± 0.037, and 0.883 ± 0.037 mean$\pm$ standard deviation for

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

(a) Sample Outputs obtained from ResNet50

(b) Sample Outputs obtained from MobileNetV2

(c) Sample Outputs obtained from EfficientNet-B0

(d) Non-Gun Gesture-1

(e) Gun Gesture-1

(f) Non-Gun Gesture-2

(g) Gun Gesture-2

Fig. 10 Detected hand gestures of Gun and Non-Gun images

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

(a) Pose4Gun: Example-1 (b) Pose4Gun: Example-2

(c) Pose4Gun: Example-3 (d) Pose4Gun: Example-4

Fig. 11 Detected guns (small firearms) and hand types from hand poses only

accuracy, AUC, and F1-Score, respectively. Irrespective of the feature selection techniques, DNN achieves 0.916, 0.950, and 0.921 in mACC, mAUC, and mF1-Score, respectively. The 2nd best performance is obtained from FDFS and XGB combination. However, the gap between the "FDFS+DNN" and "FDFS+XGB" is significant. Classifier-wise means and feature selection technique-wise mean and standard deviation are computed to identify the best-performing combination for Pose4Gun detection. It is found that the DNN as a classifier and FDFS as a feature selection scheme outperform others.

6 Conclusion

A pose-estimated dataset is modeled using machine learning and combined with a new fuzzy discernible feature selection technique. The dataset has also been examined and validated with other popular machine learning algorithms and different evolutionary-based, population-based, and rough-set-based feature selection techniques. However, the former outperforms the traditional alternatives in terms of accuracy and time taken for feature selection. The proposed hand pose estimated keypoints provide an essential gun-holding pattern that is discriminating over other hand gestures (poses). Based on the accuracy and inference time ($\sim$0.0001 sec.), the second-best performance is achieved at 89% and 82% accuracies using metric learning with FDFS and without FDFS, respectively. Furthermore, the best accuracy results from DNN are 93% and 89%, with FDFS and without FDFS, respectively. Detecting small firearms (pistols or revolvers) is a novel and fast alternative to deep learning-based object detection methods. The first adaption of the pose4gun pipeline has performed quite satisfactorily.

However, the study can be extended with more diverse firearms. Furthermore, firearms detection can be combined with full-body pose estimation to develop a more comprehensive automatic classification of violent and non-violent activities in the public space.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 19 Table of Tables to demonstrate a rigorous comparative analysis among different used learners and feature selection techniques

Model	ACC					mACC	AUC					mAUC	F1-SCORE					mF1-Score
	QR	GA	PS	GS	FD		QR	GA	PS	GS	FD		QR	GA	PS	GS	FD	
LOGREG	0.80	0.79	0.84	0.81	0.81	0.810	0.87	0.86	0.88	0.88	0.89	0.876	0.81	0.79	0.81	0.82	0.79	0.806
KNN	0.81	0.79	0.78	0.81	0.86	0.810	0.85	0.87	0.86	0.85	0.88	0.862	0.84	0.82	0.81	0.84	0.87	0.836
DTREE	0.84	0.84	0.85	0.77	0.87	0.834	0.84	0.84	0.83	0.76	0.86	0.826	0.85	0.84	0.85	0.79	0.88	0.842
METRIC	0.84	0.84	0.80	0.80	0.89	0.834	0.94	0.93	0.93	0.91	0.93	0.928	0.86	0.86	0.84	0.82	0.90	0.856
XGB	0.89	0.89	0.88	0.87	0.89	0.884	0.93	0.96	0.93	0.94	0.93	0.938	0.90	0.91	0.88	0.88	0.90	0.894
DNN	0.92	0.92	0.91	0.90	0.93	**0.916**	0.95	0.95	0.95	0.94	0.96	**0.950**	0.92	0.93	0.91	0.89	0.93	**0.921**
MEAN	0.850	0.845	0.843	0.826	**0.875**	X	0.896	0.901	0.896	0.880	**0.908**	X	0.863	0.859	0.857	0.840	**0.883**	X
STD. DEV.	0.046	0.052	0.048	0.048	**0.039**	X	0.048	0.051	0.047	0.068	**0.037**	X	0.039	0.052	0.044	0.038	**0.037**	X

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Data Availability The datasets generated during and/or analysed during the current study are available in the Face and Gun Dataset repository, https://github.com/cserajdeep/Face-and-Gun-Dataset

Declarations

Conflicts of interest The authors declare that they have no conflict of interest.

Ethical Approval Not Applicable.

Informed consent Informed consent was obtained from all individual participants included in the study.

References

1. Gun Violence – Key Facts (2018) https://www.amnesty.org/en/what-we-do/arms-control/gun-violence/. Accessed: 26 Jun 2022
2. There have been over 250 mass shootings so far in (2022) https://www.washingtonpost.com/nation/2022/06/02/mass-shootings-in-2022/. Accessed: 16 Jun 2022
3. What the data says about gun deaths in the U.S. (2022) https://www.pewresearch.org/fact-tank/2022/02/03/what-the-data-says-about-gun-deaths-in-the-u-s/. Accessed: 10 May 2022
4. Congress passes first gun control bill in decades (2022). https://www.bbc.com/news/world-us-canada-61919752. Accessed: 28 Jun 2022
5. US passes gun-control bill following rare bipartisan compromise (2022) https://www.ft.com/content/b8a1fc16-812f-443e-aa06-e14ab8a6501d. Accessed: 28 Jun 2022
6. Sidhu Moose Wala: What we know about India rapper's murder (2022). https://www.bbc.com/news/world-asia-india-61862038. Accessed: 28 Jun 2022
7. Kruegle H (2011) CCTV Surveillance: Video practices and technology. https://www.amazon.in/CCTV-Surveillance-Video-Practices-Technology/dp/0750677686
8. Trottier D (2014) Crowdsourcing CCTV surveillance on the Internet. Inf Commun Soc 17(5):609–626. https://doi.org/10.1080/1369118X.2013.808359
9. Targ S, Almeida D, Lyman K (2016) Resnet in resnet: Generalizing residual architectures. arXiv:1603.08029. https://doi.org/10.48550/arXiv.1603.08029
10. Efficientnet (2019) https://keras.io/api/applications/efficientnet/. Accessed: 20 May 2022
11. Harshvardhan G, Sahu A, Gourisaria MK, Singh PK, Hong WC, Singh V, Balabantaray BK (2022) On the dynamics and feasibility of transferred inference for diagnosis of invasive ductal carcinoma: A perspective. IEEE Access 10:30870–30889. https://doi.org/10.1109/ACCESS.2022.3159700
12. Chandra S, Gourisaria MK, Gm H, Konar D, Gao X, Wang T, Xu M (2022) Prolificacy Assessment of spermatozoan via state-of-the-art deep learning frameworks. IEEE Access 10:13715–13727. https://doi.org/10.1109/ACCESS.2022.3146334
13. Ren S, He K, Girshick R, Sun J (2015) Faster R-CNN: Towards real-time object detection with region proposal networks. https://ieeexplore.ieee.org/book/6267330
14. Jiang P, Ergu D, Liu F, Cai Y, Ma B (2022) A Review of YOLO algorithm developments. Procedia Comput Sci 199:1066–1073. https://doi.org/10.1016/j.procs.2022.01.135
15. Olmos R, Tabik S, Herrera F (2018) Automatic handgun detection alarm in videos using deep learning. Neurocomputing 275:66–72. https://doi.org/10.1016/j.neucom.2017.05.012
16. Egiazarov A, Mavroeidis V, Zennaro FM, Vishi K (2019) Firearm detection and segmentation using an ensemble of semantic neural networks. In: 2019 European Intelligence and Security Informatics Conference (EISIC), pp 70–77. https://doi.org/10.1109/EISIC49498.2019.9108871
17. Chatterjee R, Chatterjee A, Islam S, Khan MK (2022) An object detection-based few-shot learning approach for multimedia quality assessment. Multimedia Systems, pp 1–14. https://doi.org/10.1007/s00530-021-00881-8
18. Ruiz-Santaquiteria J, Velasco-Mata A, Vallez N, Bueno G, Álvarez-García JA, Deniz O (2021) Handgun detection using combined human pose and weapon appearance. IEEE Access 9:123815–123826. https://doi.org/10.1109/ACCESS.2021.3110335
19. Lamas A, Tabik S, Montes AC, Pérez-Hernández F, García J, Olmos R, Herrera F (2022) Human pose estimation for mitigating false negatives in weapon detection in video-surveillance. Neurocomputing. https://doi.org/10.1016/j.neucom.2021.12.059

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

20. Velasco-Mata A, Ruiz-Santaquiteria J, Vallez N, Deniz O (2021) Using human pose information for handgun detection. Neural Comput Appl 33(24):17273–17286. https://doi.org/10.1007/s00521-021-06317-8
21. Zhang F, Bazarevsky V, Vakunov A, Tkachenka A, Sung G, Chang CL, Grundmann M (2020) Mediapipe hands: On-device real-time hand tracking. arXiv:2006.10214. https://doi.org/10.48550/arXiv.2006.10214
22. Duy Khuat B, Thai Phung D, Thi Thu Pham H, Ngoc Bui A, Tung Ngo S (2021) Vietnamese sign language detection using Mediapipe. In: 2021 10th International conference on software and computer applications, pp 162–165. https://doi.org/10.1145/3457784.3457810
23. Lugaresi C, Tang J, Nash H, McClanahan C, Uboweja E, Hays M, Zhang F, Chang CL, Yong MG, Lee J, et al (2019) Mediapipe: A framework for building perception pipelines. arXiv:1906.08172. https://doi.org/10.48550/arXiv.1906.08172
24. Martin-Bautista MJ, Vila MA (1999) A survey of genetic feature selection in mining issues. In: Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406) vol 2, pp 1314–1321. https://doi.org/10.1109/CEC.1999.782599
25. Babatunde OH, Armstrong L, Leng J, Diepeveen D (2014) A genetic algorithm-based feature selection. https://ro.ecu.edu.au/ecuworkspost2013/653/
26. Sastry K, Goldberg D, Kendall G (2005) Genetic algorithms. Search methodologies, pp 97–125. https://doi.org/10.1007/0-387-28356-0_4
27. Mirjalili S (2019) Genetic algorithm. Evolutionary algorithms and neural networks, pp. 43–55. https://doi.org/10.1007/978-3-319-93025-1
28. Liu Y, Wang G, Chen H, Dong H, Zhu X, Wang S (2011) An improved particle swarm optimization for feature selection. J Bionic Eng 8(2):191–200. https://doi.org/10.1016/S1672-6529(11)60020-6
29. Eberhart R, Kennedy J (1995) Particle Swarm Optimization. In: Proceedings of the IEEE international conference on neural networks vol 4, pp 1942–1948. https://doi.org/10.1109/ICNN.1995.488968
30. Marini F, Walczak B (2015) Particle swarm optimization (PSO). A tutorial. Chemometr Intell Lab Syst 149:153–165. https://doi.org/10.1016/j.chemolab.2015.08.020
31. Papa JP, Pagnin A, Schellini SA, Spadotto A, Guido RC, Ponti M, Chiachia G, Falcão AX (2011) Feature selection through gravitational search algorithm. 2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp 2052–2055. https://doi.org/10.1109/ICASSP.2011.5946916
32. Taradeh M, Mafarja M, Heidari AA, Faris H, Aljarah I, Mirjalili S, Fujita H (2019) An evolutionary gravitational search-based feature selection. Inf Sci 497:219–239. https://doi.org/10.1016/j.ins.2019.05.038
33. Rashedi E, Nezamabadi-Pour H, Saryazdi S (2009) GSA: a gravitational search algorithm. Inf Sci 179(13):2232–2248. https://doi.org/10.1016/j.ins.2009.03.004
34. Rashedi E, Nezamabadi-Pour H, Saryazdi S (2010) BGSA: binary gravitational search algorithm. Nat Comput 9(3):727–745. https://doi.org/10.1007/s11047-009-9175-3
35. Komorowski J, Pawlak Z, Polkowski L, Skowron A (1999) Rough sets: A tutorial. http://www2.cs.uregina.ca/ziarko/cs836-08/RS-tutorial.pdf
36. Pawlak Z (2002) Rough set theory and its applications. Journal of Telecommunications and information technology pp 7–10. https://doi.org/10.1080/019697298125470
37. Chatterjee R, PLP M (2011) Rough set & it's variants: Variable precision rough set and fuzzy rough set approaches. https://doi.org/10.13140/2.1.1354.9447
38. Thangavel K, Velayutham C (2012) Rough set based unsupervised feature selection in digital mammogram image using entropy measure. In: 2012 International Conference on Biomedical Engineering (ICoBE), pp 10–16. https://doi.org/10.1109/ICoBE.2012.6178946
39. Jensen R, Shen Q (2008) Computational intelligence and feature selection: Rough and fuzzy approaches. https://ieeexplore.ieee.org/book/5236578
40. Jović A, Brkić K, Bogunović N (2015) A review of feature selection methods with applications. In: 2015 38th international convention on information and communication technology, electronics and microelectronics (MIPRO), pp 1200–1205. https://doi.org/10.1109/MIPRO.2015.7160458
41. Asai M, McAleer M, Yu J (2006) Multivariate stochastic volatility: a review. Econ Rev 25(2–3):145–175. https://doi.org/10.1080/07474930600713564
42. Pawlak Z (1982) Rough sets. Int J Comput Inf Sci 11(5):341–356. https://doi.org/10.1007/BF01001956
43. Chatterjee R, Sanyal DK, Chatterjee A (2021). Fuzzy-discernibility matrix-based efficient feature selection techniques for improved motor-imagery EEG signal classification. https://doi.org/10.1101/2021.03.24.436722
44. Chatterjee R, Maitra T, Islam SH, Hassan MM, Alamri A, Fortino G (2019) A novel machine learning based feature selection for motor imagery EEG signal classification in Internet of medical things environment. Future Gener Comput Syst 98:419–434. https://doi.org/10.1016/j.future.2019.01.048

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

45. Chatterjee R, Guha D, Sanyal DK, Mohanty SN (2016) Discernibility matrix based dimensionality reduction for EEG signal. In: 2016 IEEE Region 10 Conference (TENCON), pp 2703–2706. https://doi.org/10.1109/TENCON.2016.7848530
46. Chatterjee R, Bandyopadhyay T, Sanyal DK, Guha D (2017) Dimensionality reduction of EEG signal using fuzzy discernibility matrix. In: 2017 10th International Conference on Human System Interactions (HSI) pp 131–136. https://doi.org/10.1109/HSI.2017.8005014
47. Kumar N, Yadav SK, Yadav DS (2020) Similarity measure approaches applied in text document clustering for information retrieval. In: 2020 6th International Conference on Parallel, Distributed and Grid Computing (PDGC), pp 88–92. https://doi.org/10.1109/PDGC50313.2020.9315851
48. Montavon G, Samek W, Müller KR (2018) Methods for interpreting and understanding deep neural networks. Digit Signal Process 73:1–15. https://doi.org/10.1016/j.dsp.2017.10.011
49. Miikkulainen R, Liang J, Meyerson E, Rawal A, Fink D, Francon O, Raju B, Shahrzad H, Navruzyan A, Duffy N, et al (2019) Evolving deep neural networks. In: Artificial intelligence in the age of neural networks and brain computing, pp 293–312. https://doi.org/10.1016/B978-0-12-815480-9.00015-3
50. Sze V, Chen YH, Yang TJ, Emer JS (2017) Efficient processing of deep neural networks: A tutorial and survey. Proc IEEE 105(12):2295–2329. https://doi.org/10.1109/JPROC.2017.2761740
51. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 770–778. https://doi.org/10.1109/CVPR.2016.90
52. Tan M, Le QV (2019) Efficientnet: Rethinking model scaling for convolutional neural networks. arXiv:1905.11946. https://doi.org/10.48550/arXiv.1905.11946
53. Wu Z, Shen C, Van Den Hengel A (2019) Wider or deeper: Revisiting the resnet model for visual recognition. Pattern Recognit 90:119–133. https://doi.org/10.1016/j.patcog.2019.01.006
54. Jain P, Khare S, Gourisaria MK (2021) A Data Mining Solution to Predict Campus Placement. In: 2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON) pp 1–7. https://doi.org/10.1109/GUCON50781.2021.9573551
55. Internet movie firearms database (2008) http://www.imfdb.org/wiki/Main_Page. Accessed: 17 May 2021

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Rajdeep Chatterjee[1,2] · Ankita Chatterjee[2]

Ankita Chatterjee
ankita.datta24@gmail.com

1 School of Computer Engineering, Kalinga Institute of Industrial Technology (Deemed to be University), Patia, Bhubaneswar 751024, Odisha, India

2 Amygadala AI, Bhubaneswar, Odisha, India

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com